

Map vs Unordered_Map: Complete Guide

Core Difference

Feature	Map	Unordered_Map
Internal Structure	Binary Search Tree (Red-Black Tree)	Hash Table
Ordering	Maintains sorted order	No ordering
Time Complexity	O(log n) guaranteed	O(1) average, O(n) worst case
Key Requirements	Comparison operator (<code><</code>)	Hash function

Key Type Limitations

Map - Works with ANY comparable type:

```
cpp
map<int, string> m1; ..... ✓ Works
map<pair<int, int>, string> m2; ..... ✓ Works
map<vector<int>, string> m3; ..... ✓ Works
map<MyCustomClass, string> m4; ..... ✓ Works (if < operator defined)
```

Unordered_Map - Limited to types with hash functions:

```
cpp
unordered_map<int, string> um1; ..... ✓ Works (built-in hash)
unordered_map<string, string> um2; ..... ✓ Works (built-in hash)
unordered_map<pair<int, int>, string> um3; ..... ✗ ERROR! No built-in hash
unordered_map<vector<int>, string> um4; ..... ✗ ERROR! No built-in hash
```

Why This Limitation Exists

Map only needs COMPARISON:

```
cpp
// For pair<int, int> comparison:
pair<int, int> a = {1, 2};
pair<int, int> b = {1, 3};
bool result = (a < b); // Built-in: compares first, then second
// Result: true (because 2 < 3)
```

Unordered_Map needs HASH FUNCTIONS:

```
cpp

// For pair<int, int> hashing - NO built-in way!
pair<int, int> p = {1, 2};
size_t hash_value = hash<pair<int, int>>()(p); // ❌ Doesn't exist!

// We need to define our own:
struct PairHash {
    size_t operator()(const pair<int, int>& p) const {
        return hash<int>()(p.first) ^ (hash<int>()(p.second) << 1);
    }
};

unordered_map<pair<int, int>, int, PairHash> um; ✅ Now works!
```

Internal Working

Map (Binary Search Tree):

Tree structure for map<int, string>:

```
..... 5
..... / \
..... 3   8
..... / \ / \
..... 1  4 7 9
```

// To find key 7: traverse tree path, $O(\log n)$ time

Unordered_Map (Hash Table):

Hash table structure:

Bucket 0: []

Bucket 1: [key: 15, value: "fifteen"] ← $\text{hash}(15) \% 10 = 5 \rightarrow$ collision → bucket 1

Bucket 2: []

Bucket 3: [key: 3, value: "three"] ← $\text{hash}(3) \% 10 = 3$

Bucket 4: []

Bucket 5: [key: 5, value: "five"] ← $\text{hash}(5) \% 10 = 5$

...

// To find key 5: $\text{hash}(5) \rightarrow$ bucket 5 → direct access, $O(1)$ time

Performance Comparison

Operation	Map	Unordered_Map
Insert	$O(\log n)$	$O(1)$ average, $O(n)$ worst
Search	$O(\log n)$	$O(1)$ average, $O(n)$ worst
Delete	$O(\log n)$	$O(1)$ average, $O(n)$ worst
Iterate in order	$O(n)$ ✓	$O(n)$ ✗ (unordered)
Range queries	$O(\log n + k)$ ✓	Not supported ✗

Key Takeaways

Why doesn't unordered_map just use trees?

Answer: Because then it would lose its $O(1)$ advantage and become identical to map! The whole point of unordered_map is to sacrifice ordering for speed.

Why is hashing harder than comparison?

Comparison: Just define an order relationship ($a < b$?) **Hashing:** Must create a function that:

- Distributes values evenly across buckets
- Is deterministic and fast
- Handles collisions well
- Maps infinite input space to finite bucket space

When to use which?

Use Map when:

- You need sorted data
- You need range queries
- Key type doesn't have hash function
- You want guaranteed $O(\log n)$ performance

Use Unordered_Map when:

- You want fastest possible lookup/insertion
- You don't care about ordering
- Key type has hash function (int, string, etc.)

- You can tolerate worst-case $O(n)$ scenarios

Built-in Hash Functions Available

cpp

// These work with unordered_map out of the box:

`unordered_map<int, T>.....` // 

`unordered_map<long, T>.....` // 

`unordered_map<double, T>.....` // 

`unordered_map<string, T>.....` // 

`unordered_map<char, T>.....` // 

// These DON'T work without custom hash:

`unordered_map<pair<int,int>, T>.....` // 

`unordered_map<vector<int>, T>.....` // 

`unordered_map<set<int>, T>.....` // 

`unordered_map<CustomStruct, T>.....` // 